

COMP1004

Databases and Interfaces

Module conveners:

Nazia Hameed, Kai Xu

Today's lecture

- Structure query language(SQL)
- Creating tables
- Datatypes
- Constraints
- Alter table
- Drop table
- Insert data in tables
- Update data in tables
- Delete data from tables
- Select

Structure query language(SQL)

- SQL is a standard language for accessing and manipulating databases
- SQL became a standard of the
 - American National Standards Institute (ANSI) in 1986
 - International Organization for Standardization (ISO) in 1987.
- SQL can
 - execute queries against a database
 - retrieve data from a database
 - insert records in a database
 - update records in a database
 - delete records from a database
 - create new databases
 - create new tables in a database
 - alter/drop tables in a database

Structure query language(SQL)

- Keep in mind that while SQL is a standard
- It is not supported exactly the same way by all DBMSs.
- In practice, you may need to update your SQL queries to work with different DBMSs
- SQL is used in the many relational DBMS
 - MySQL
 - PostgreSQL
 - Oracle DB
 - SQL Server
 - **SQLite**
 - MS Access

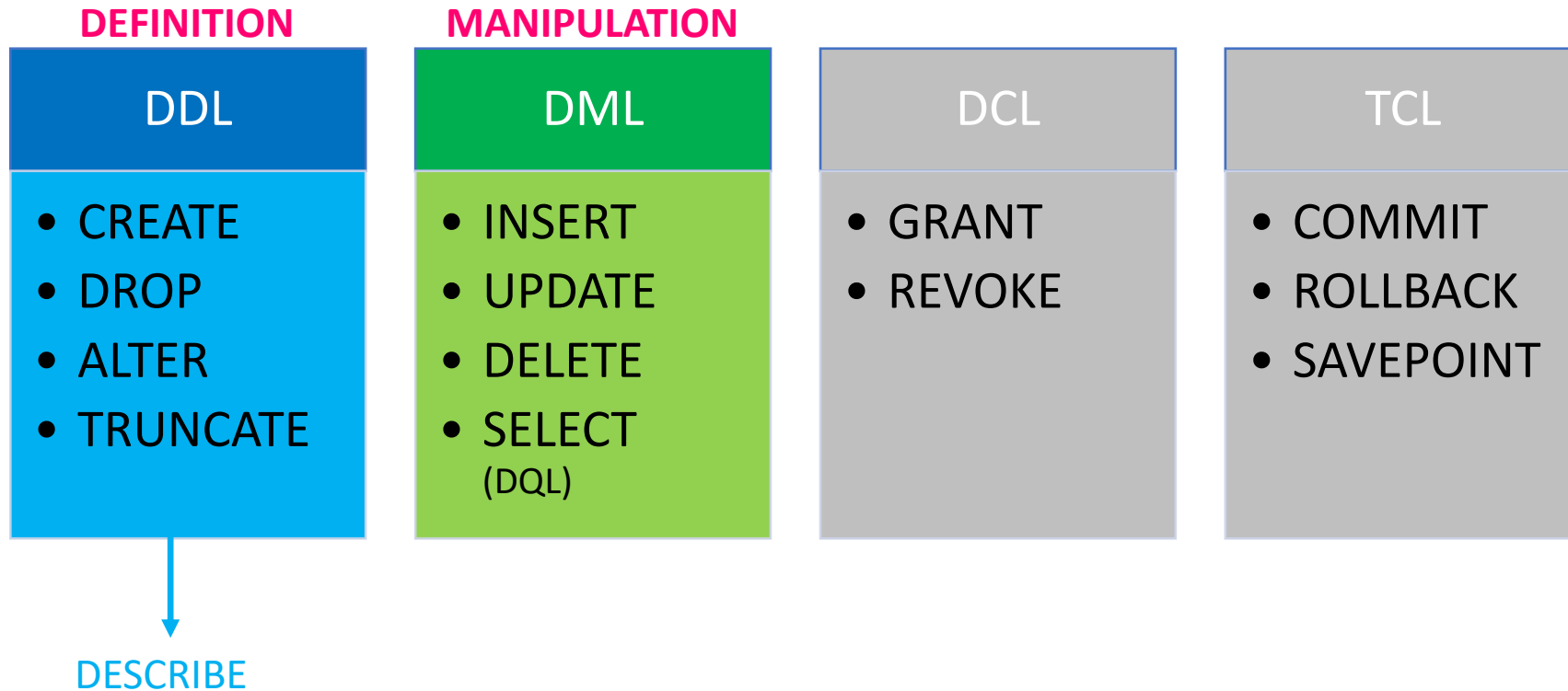
SQL Commands

DDL: Data Definition Language
DML: Data Manipulation Language
DCL: Data Control Language
TCL: Transaction Control Language

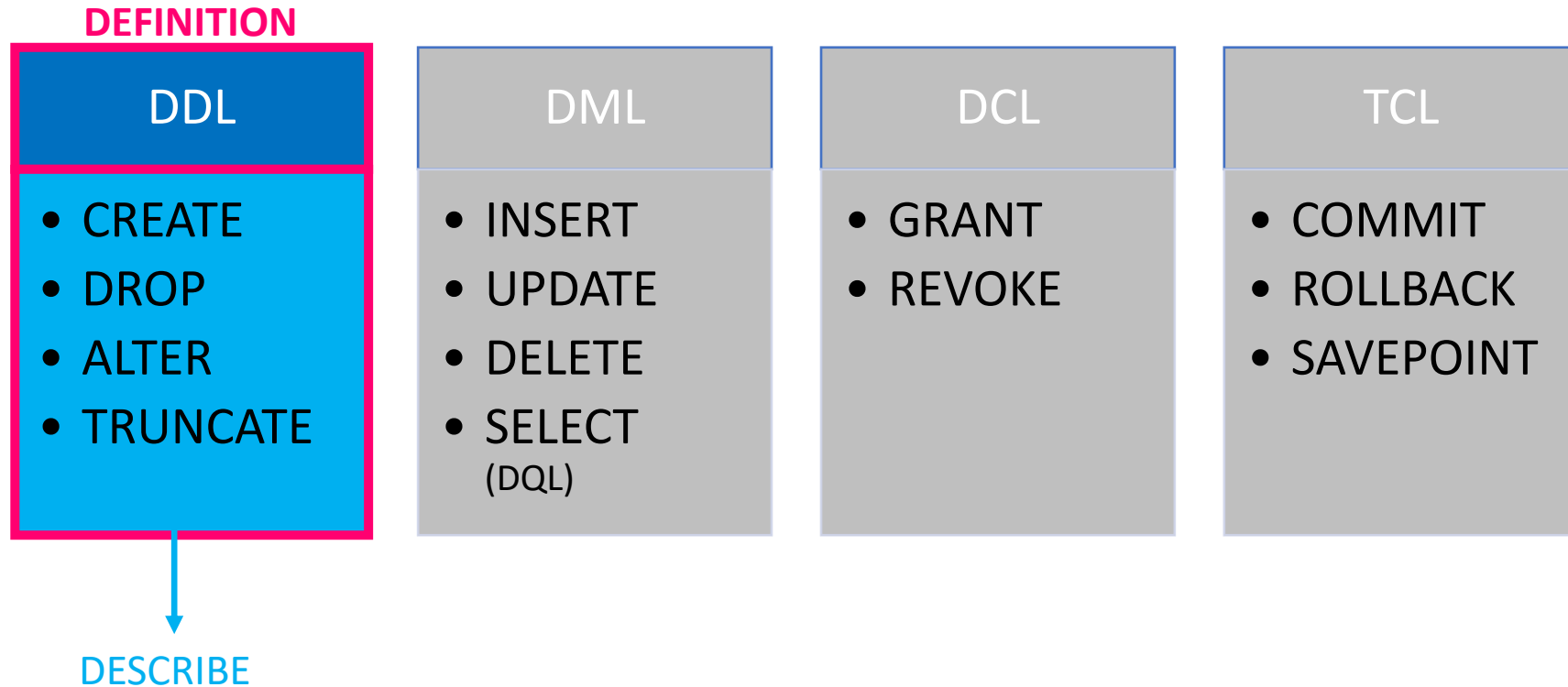
DDL	DML	DCL	TCL
• CREATE • DROP • ALTER • TRUNCATE	• INSERT • UPDATE • DELETE • SELECT	• GRANT • REVOKE	• COMMIT • ROLLBACK • SAVEPOINT

SQL Commands

DDL: Data Definition Language
DML: Data Manipulation Language
DCL: Data Control Language
TCL: Transaction Control Language



SQL Commands



Data types in SQLite

- SQLite provides five primitive data types which are referred to as **storage classes**.
- A storage class is more general than a datatype
- SQLite does not have a separate Boolean storage class. Instead, Boolean values are stored as integers 0 (false) and 1 (true).

SQLite storage classes

- NULL
 - The value is a NULL value.
- INTEGER
 - The value is a signed integer, stored in 1, 2, 3, 4, 6, or 8 bytes depending on the magnitude of the value.
- REAL
 - The value is a floating point value
- TEXT
 - The value is a text string
- BLOB
 - The value is a blob of data, stored exactly as it was input.
- Limits in SQLite

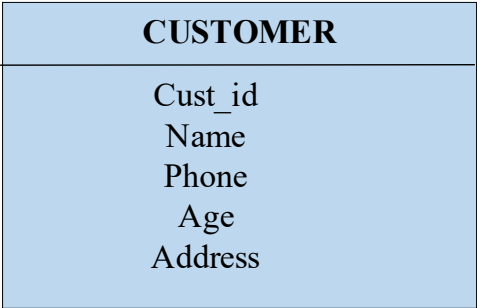
Creating tables: The formal syntax

- Requires:
 - The **name** of the table
 - List of **column definitions**
 - List of **constraints** (optional)
 - Primary keys
 - Foreign keys
- **Table names** need to be unique in the database
- **Column names** need to be unique in the table

```
CREATE TABLE <name>
(
    <col-def-1>,
    <col-def-2>,
    ...
    <col-def-n>,
    <constraint-1>,
    ...
    <constraint-k>
)
```

Creating tables

Create table customer(
Cust_id integer,
Name text,
Phone text,
Age integer,
Address text)



customer	CREATE TABLE customer(Cust_id integer not null, Name text not null, Phone text not null, Age integer null, Address text not null)	
Cust_id	integer	"Cust_id" integer NOT NULL
Name	text	"Name" text NOT NULL
Phone	text	"Phone" text NOT NULL
Age	integer	"Age" integer
Address	text	"Address" text NOT NULL

- Table structure can be viewed using
 - PRAGMA table_info(customer);

	cid	name	type	notnull	dflt_value	pk
1	0	Cust_id	integer	0	NULL	0
2	1	Name	text	0	NULL	0
3	2	Phone	text	0	NULL	0
4	3	Age	integer	0	NULL	0
5	4	Address	text	0	NULL	0

Drop tables

- You can delete tables with the DROP keyword:
 - DROP TABLE [IF EXISTS] table-name;
- For example: DROP TABLE IF EXISTS customer;
- Foreign Key constraints will prevent you from deleting a table if it is referenced by another table.
 - You can delete the referencing table first, then the referenced table
- **Caution:**
 - Be very careful with this command. It will delete the table and all data. There is no undo

Constraint

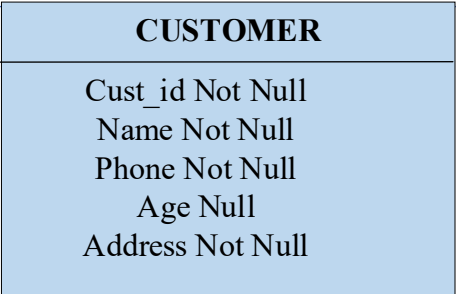
- NOT NULL
 - Ensures that a column cannot have NULL value.
- DEFAULT
 - Sets a default value for a column if no value is specified
- UNIQUE
 - Ensures that all values in a column are different
- CHECK
 - Ensures that all values in a column satisfies certain conditions
- PRIMARY Key
 - Uniquely identifies each row/record in a database table

Null

- NULLs represent missing information
- NOT NULL columns cannot be NULL
- If neither NULL nor NOT NULL are specified, NULL-able is assumed
- NULL/NOT NULL are a domain restriction
 - You should think about what being NULL-able means for each column, and its impact on the system
 - Can a name be NULL? (big impact)
 - Can a date of birth be NULL? (smaller impact)
 - Etc.
 - You should think about what not being NULL-able means for each column as well!
 - E.g., should a user not exist because they don't want to enter their date of birth?
 - It's a design decision!

Creating tables

Create table customer(
Cust_id integer not null,
Name text not null,
Phone text not null,
Age integer null,
Address text not null)



customer	CREATE TABLE customer(Cust_id integer, Name text, Phone text, Age integer, Address text)	
Cust_id	integer	"Cust_id" integer
Name	text	"Name" text
Phone	text	"Phone" text
Age	integer	"Age" integer
Address	text	"Address" text

- Table structure can be viewed using
 - PRAGMA table_info(customer);

	cid	name	type	notnull	dflt_value	pk
1	0	Cust_id	integer	1	NULL	0
2	1	Name	text	1	NULL	0
3	2	Phone	text	1	NULL	0
4	3	Age	integer	0	NULL	0
5	4	Address	text	1	NULL	0

Unique

- Ensures that all values in a column are different
- Do not allow the same value to be inserted multiple times in this column
- You can have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table
- Does not have to be a primary key
- E.g., your phone username should be unique but is not necessarily the primary key used by the university systems

```
Create table customer(  
  Cust_id integer not null,  
  Name text not null,  
  Phone text not null unique,  
  Age integer null,  
  Address text not null)
```


Default

- Allows you to provide a default value if not provided when inserting new data in the table
- E.g., a new student would have a default semester of 1, unless specified otherwise

```
Create table customer(  
  Cust_id integer not null,  
  Name text not null,  
  Phone text not null,  
  Age integer null,  
  Address text not null,  
  Orderno integer notnull default 0)
```



CUSTOMER
Cust_id Not Null
Name Not Null
Phone Not Null
Age Null
Address Not Null
Orderno Not Null default 0

Check

- The CHECK constraint is used to limit the value range that can be placed in a column
- If you define a CHECK constraint on a column it will allow only certain values for this column

```
Create table customer(  
  Cust_id integer not null,  
  Name text not null,  
  Phone text not null,  
  Age integer check(Age >10),  
  Address text not null  
)
```



CUSTOMER	
<u>Cust_id</u>	
Name	Not Null
Phone	Not Null
Age	Check Age >10
Address	Not Null

The importance of keys

- A key identifies a row
- Therefore it must be **UNIQUE** and **NOT NULL**
- A key can be one or more column
- Keys are what keep the relational model running

Primary keys

- The PRIMARY KEY constraint uniquely identifies each record in a table
- Primary keys must contain UNIQUE values, and cannot contain NULL values
- A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields)
- If there is only one primary key

```
Create table customer(  
  Cust_id integer primary key,  
  Name text not null,  
  Phone text not null,  
  Age integer check(Age > 10),  
  Address text not null  
)
```

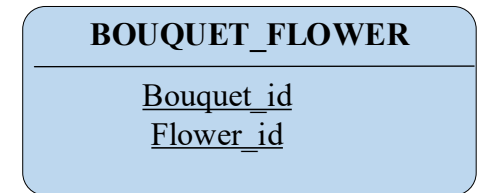


CUSTOMER	
<u>Cust_id</u>	
Name	Not Null
Phone	Not Null
Age	Null
Address	Not Null

Primary keys

- If primary key consists of two or more columns i.e. composite, we use PRIMARY KEY table constraint to define the key

```
Create table bouquet_flower(  
  Bouquet_id integer,  
  Flower_id integer,  
  Primary key(Bouquet_id, Flower_id))
```



FOREIGN KEYS

- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables
- A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table
- The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table
- Require:
 - The columns that make up the foreign key
 - The table that is referenced by the foreign key
 - The column(s) in that table that are referenced by the foreign key

```
FOREIGN KEY (col1, col2, ...) REFERENCES table  
(refcol1, refcol2,...)
```

FOREIGN KEYS

Create table bouquets(

Bouquet_id integer primary key,

Bouquet_name text,

Price Real)

Create table flowers(

Flower_id integer primary key,

Flower_name text)

Create table bouquet_flower(

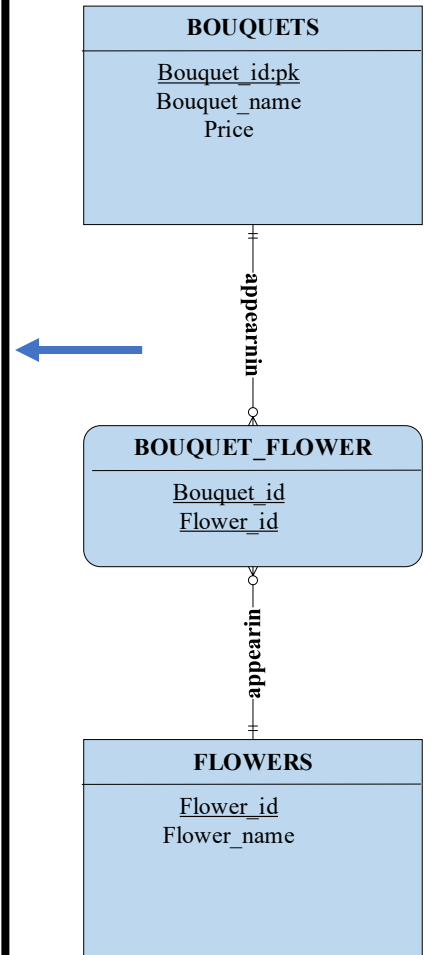
Bouquet_id integer,

Flower_id integer,

Primary key(Bouquet_id, Flower_id),

Foreign key(Bouquet_id) REFERENCES bouquets(Bouquet_id),

Foreign key(Flower_id) REFERENCES flowers(Flower_id))



SQLite foreign key constraint actions

- What would happen if you delete a row in the flowers or bouquets table?
- Should all the corresponding rows in the bouquet_flower table are also deleted?
- To specify how foreign key constraint behaves whenever the parent key is deleted or updated, you use the ON DELETE or ON UPDATE action as follows:

```
FOREIGN KEY (foreign_key_columns)
REFERENCES
parent_table(parent_key_columns)
    ON UPDATE action
    ON DELETE action
```


SQLite foreign key constraint actions

- SQLite supports the following actions:
- **SET NULL**
 - When the parent key delete or update, the corresponding child keys of all rows in the child table set to NULL.
- **SET DEFAULT**
 - The SET DEFAULT action sets the value of the foreign key to the default value specified in the column definition when you create the table.
- **RESTRICT**
 - The RESTRICT action does not allow you to change or delete values in the parent key of the parent table.
- **NO ACTION**
 - The NO ACTION does not mean by-pass the foreign key constraint.
- **CASCADE**
 - The CASCADE action propagates the changes from the parent table to the child table when you update or delete the parent key.

Setting NULL

```
Create table bouquet_flower(  
Bouquet_id integer,  
Flower_id integer,  
Primary key(Bouquet_id, Flower_id),  
Foreign key(Bouquet_id) REFERENCES bouquets(Bouquet_id)  
ON UPDATE SET NULL  
ON DELETE SET NULL,  
Foreign key(Flower_id) REFERENCES flowers(Flower_id)  
ON UPDATE SET NULL  
ON DELETE SET NULL)
```

Setting CASCADE

```
Create table bouquet_flower(  
Bouquet_id integer,  
Flower_id integer,  
Primary key(Bouquet_id, Flower_id),  
Foreign key(Bouquet_id) REFERENCES bouquets(Bouquet_id)  
ON UPDATE CASCADE  
ON DELETE CASCADE,  
Foreign key(Flower_id) REFERENCES flowers(Flower_id)  
ON UPDATE CASCADE  
ON DELETE CASCADE)
```

SQLite AUTOINCREMENT

- Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table
- Often this is the primary key field that we would like to be created automatically every time a new record is inserted

```
Create table customer(  
  Cust_id integer primary key AUTOINCREMENT,  
  Name text not null,  
  Phone text not null,  
  Age integer check(Age >10),  
  Address text not null  
)
```

SQLite ALTER TABLE

- SQLite supports a very limited functionality of the ALTER TABLE statement

- rename a table

```
ALTER TABLE customer RENAME TO cust;
```

- Add new column to a table

```
ALTER TABLE cust ADD COLUMN nationality;
```

- rename a column

```
ALTER TABLE cust RENAME COLUMN name TO fullname;
```

- drop column

```
ALTER TABLE cust DROP COLUMN nationality;
```

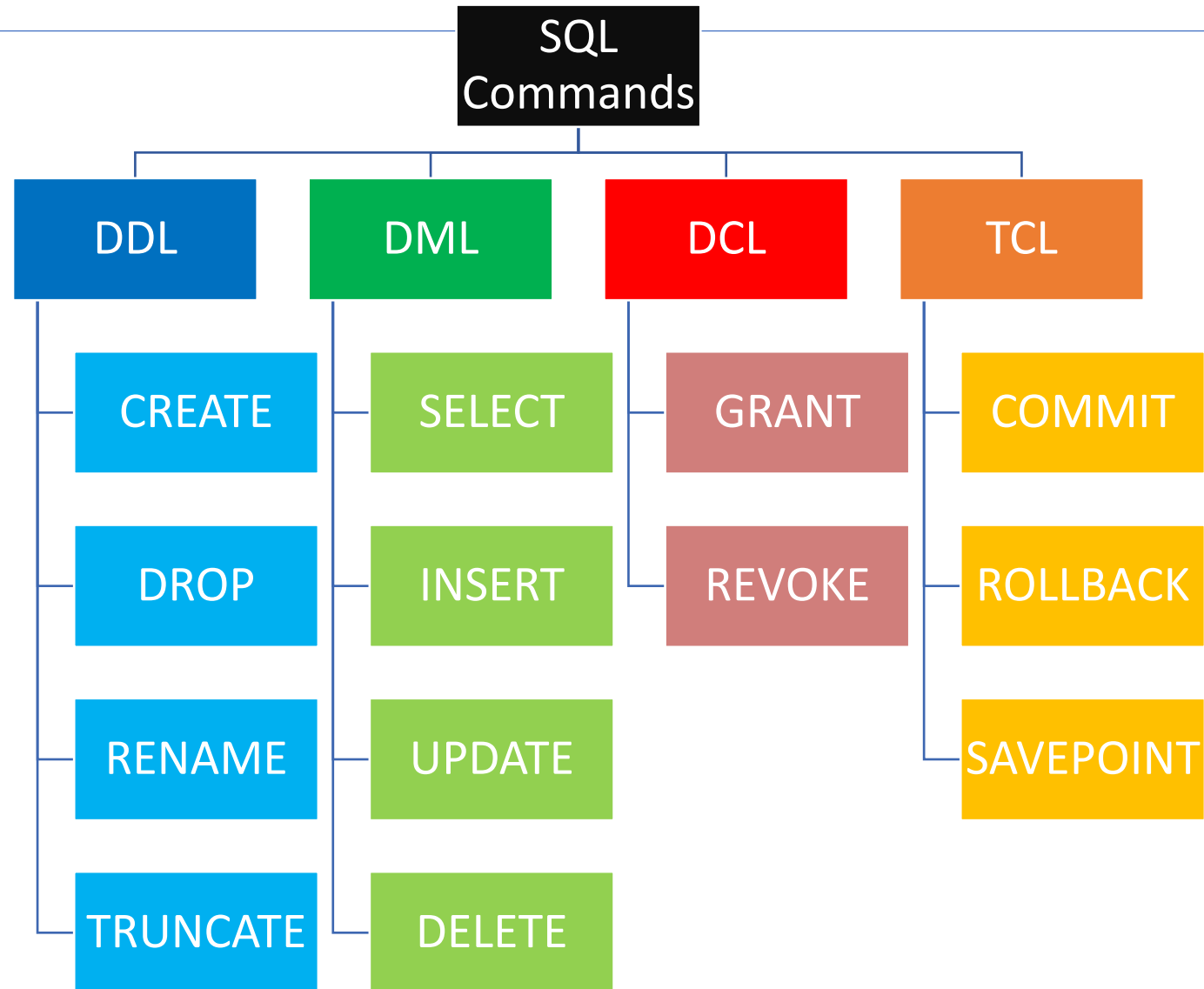
COMP1004

Databases and Interfaces

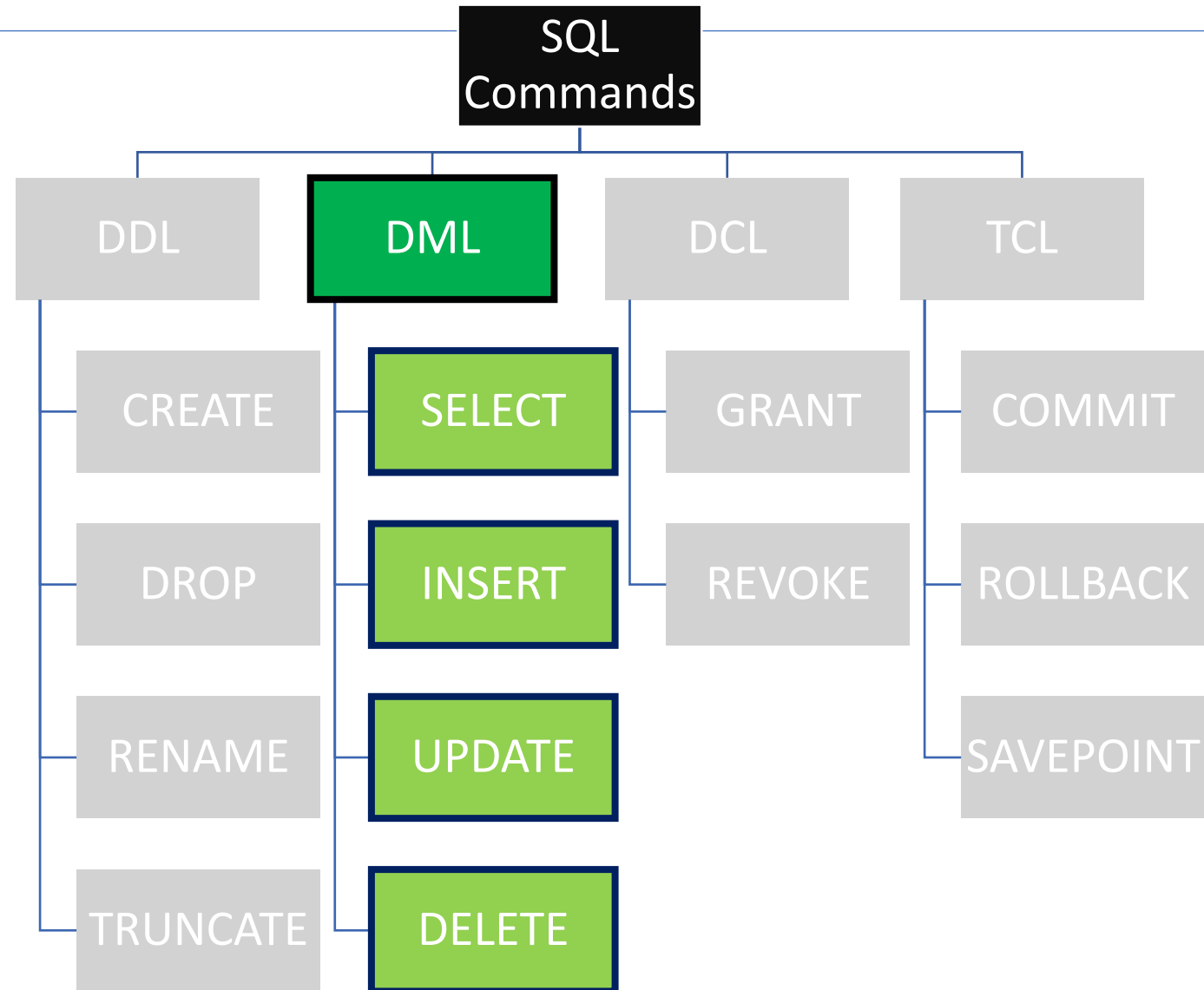
Module conveners:

Nazia Hameed, Kai Xu

DDL: Data Definition Language
DML: Data Manipulation Language
DCL: Data Control Language
TCL: Transaction Control Language



DDL: Data Definition Language
DML: Data Manipulation Language
DCL: Data Control Language
TCL: Transaction Control Language



Data manipulation language

- DML is used for inserting, retrieving, and manipulating data in a database tables
 - Insert data in tables
 - Update data in tables
 - Delete data from tables
 - Select data from tables

Insert data in tables

- To insert data into a table, you use the INSERT statement
- SQLite provides various forms of the INSERT statements
 - insert single row
 - insert multiple rows

Syntax:

```
INSERT INTO table (column1,column2 ,...)  
VALUES( value1, value2 ,...);
```

Insert data example

bouquet			CREATE TABLE bouquet(bouquet_id integer primary key autoincrement, bouquet_name text not null, price Real)
bouquet_id	integer	"bouquet_id" integer	
bouquet_name	text	"bouquet_name" text NOT NULL	
price	Real	"price" Real	

Single value

```
insert into bouquet(bouquet_name,price)
VALUES
('A New Day',23.5)
```

Multiple values

```
insert into bouquet(bouquet_name,price)
VALUES
('A New Day',23.5),
('The flower of love',20.5),
('Passion',13.0),
('I love you',33.0),
('Pretty in pink',24.5)
```

Delete data from tables

- The SQLite DELETE statement allows you to
 - delete one row
 - delete multiple rows
 - delete all rows in a table.
- Deletion of rows depends on where condition
- If no condition is specified, then all rows will be deleted

syntax

```
DELETE FROM table  
WHERE condition;
```

Delete - An illustration

	bouquet_id	bouquet_name	price
1	1	A New Day	24.0
2	2	The flower of love	20.5
3	3	Passion	13.0
4	4	I love you	33.0
5	5	Pretty in pink	24.5
6	6	You are beautiful	23.0
7	7	Amour	20.0
8	8	Simply red	26.0
9	9	Be my Valentine	13.5
10	10	Hello	10.0
11	11	Ladies Day	12.5
12	12	To mum with love	15.0
13	13	Made for mum	20.0
14	14	Heaven scent	5.0
15	15	Pastel perfection	8.0
16	16	Summer sun	22.0
17	17	Summer breeze	6.0

DELETE FROM bouquet
WHERE bouquet_id=1;



	bouquet_id	bouquet_name	price
1	2	The flower of love	20.5
2	3	Passion	13.0
3	4	I love you	33.0
4	5	Pretty in pink	24.5
5	6	You are beautiful	23.0
6	7	Amour	20.0
7	8	Simply red	26.0
8	9	Be my Valentine	13.5
9	10	Hello	10.0
10	11	Ladies Day	12.5
11	12	To mum with love	15.0
12	13	Made for mum	20.0
13	14	Heaven scent	5.0
14	15	Pastel perfection	8.0
15	16	Summer sun	22.0
16	17	Summer breeze	6.0

Delete - An illustration

	bouquet_id	bouquet_name	price
1	1	A New Day	24.0
2	2	The flower of love	20.5
3	3	Passion	13.0
4	4	I love you	33.0
5	5	Pretty in pink	24.5
6	6	You are beautiful	23.0
7	7	Amour	20.0
8	8	Simply red	26.0
9	9	Be my Valentine	13.5
10	10	Hello	10.0
11	11	Ladies Day	12.5
12	12	To mum with love	15.0
13	13	Made for mum	20.0
14	14	Heaven scent	5.0
15	15	Pastel perfection	8.0
16	16	Summer sun	22.0
17	17	Summer breeze	6.0
18	18	A New Day	23.5

DELETE FROM bouquet;



bouquet_id	bouquet_name	price
------------	--------------	-------

Update data in tables

- To update existing data in a table, you use SQLite UPDATE statement.
- If no where condition is provided all the data in the table will be updated.

syntax

```
UPDATE tablename  
SET column1 = new column1value,  
    column2 = new column2value  
WHERE  
    Condition
```

Update - An illustration

	bouquet_id	bouquet_name	price
1	1	A New Day	24.0
2	2	The flower of love	20.5
3	3	Passion	13.0
4	4	I love you	33.0
5	5	Pretty in pink	24.5
6	6	You are beautiful	23.0
7	7	Amour	20.0
8	8	Simply red	26.0
9	9	Be my Valentine	13.5
10	10	Hello	10.0
11	11	Ladies Day	12.5
12	12	To mum with love	15.0
13	13	Made for mum	20.0
14	14	Heaven scent	5.0
15	15	Pastel perfection	8.0
16	16	Summer sun	22.0
17	17	Summer breeze	6.0

UPDATE bouquet
SET price = 23.5
where
bouquet_id = 1



bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
3	Passion	13.0
4	I love you	33.0
5	Pretty in pink	24.5
6	You are beautiful	23.0
7	Amour	20.0
8	Simply red	26.0
9	Be my Valentine	13.5
10	Hello	10.0
11	Ladies Day	12.5
12	To mum with love	15.0
13	Made for mum	20.0
14	Heaven scent	5.0
15	Pastel perfection	8.0
16	Summer sun	22.0
17	Summer breeze	6.0

Update - An illustration

	bouquet_id	bouquet_name	price
1	1	A New Day	24.0
2	2	The flower of love	20.5
3	3	Passion	13.0
4	4	I love you	33.0
5	5	Pretty in pink	24.5
6	6	You are beautiful	23.0
7	7	Amour	20.0
8	8	Simply red	26.0
9	9	Be my Valentine	13.5
10	10	Hello	10.0
11	11	Ladies Day	12.5
12	12	To mum with love	15.0
13	13	Made for mum	20.0
14	14	Heaven scent	5.0
15	15	Pastel perfection	8.0
16	16	Summer sun	22.0
17	17	Summer breeze	6.0

UPDATE bouquet
SET price = 23.5



	bouquet_id	bouquet_name	price
	1	A New Day	23.5
	2	The flower of love	23.5
	3	Passion	23.5
	4	I love you	23.5
	5	Pretty in pink	23.5
	6	You are beautiful	23.5
	7	Amour	23.5
	8	Simply red	23.5
	9	Be my Valentine	23.5
	10	Hello	23.5
	11	Ladies Day	23.5
	12	To mum with love	23.5
	13	Made for mum	23.5
	14	Heaven scent	23.5
	15	Pastel perfection	23.5
	16	Summer sun	23.5
	17	Summer breeze	23.5

Database problems

- Database problems can be expressed in English, Relational Algebra and SQL
- English:
 - List all staff with a customers whose age greater than 23
- Relational Algebra:
 - $\sigma_{age > 23}(\text{customers})$
- SQL:
 - `SELECT * FROM customers WHERE age > 23;`
- You should be able to translate between any of these

Relational Algebra

- The relational algebra is a theoretical language with operations that work on one or more relations to define another relation without changing the original relation(s).
 - The five fundamental operations in relational algebra
 - Selection
 - Projection
 - Cartesian product
 - Union
 - Set difference
 - Some other operations in relational algebra
 - Join
 - Intersection

Symbols for relational algebra

Unary operations

Projection (Π / pi)

Selection (σ / sigma)

Binary operations

Cartesian product ($\times$)

Union ($\cup$)

Intersection ($\cap$)

Difference ($-$)

Join ($\bowtie$ / bowtie)

Selection

- The Selection operation works on a single relation R and defines a relation that contains only those tuples of R that satisfy the specified condition (predicate)
- $\sigma_{predicate}(R)$
- Multiple predicates can be added like this
- $\sigma_{predicate1 \text{ and } predicate2}(R)$



- Example
 - Produce a list of bouquet whose price is greater than 20.
 - $\sigma_{price > 20}(\text{bouquet})$

Selection

- $\sigma_{price > 20}(\text{bouquet})$

bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
3	Passion	13.0
4	I love you	33.0
5	Pretty in pink	24.5
6	You are beautiful	23.0
7	Amour	20.0
8	Simply red	26.0
9	Be my Valentine	13.5
10	Hello	10.0
11	Ladies Day	12.5
12	To mum with love	15.0
13	Made for mum	20.0
14	Heaven scent	5.0
15	Pastel perfection	8.0
16	Summer sun	22.0
17	Summer breeze	6.0



bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
4	I love you	33.0
5	Pretty in pink	24.5
6	You are beautiful	23.0
8	Simply red	26.0
16	Summer sun	22.0

Projection

- The Projection operation works on a single relation R and defines a relation that contains a vertical subset of R, extracting the values of specified attributes and eliminating duplicates
- $\Pi_{a,b,c\dots z}(R)$
- NOTE there is no condition in the Projection



- Example
 - Produce a list of bouquet names with their price.
 - $\Pi_{bouquet_name, price}(bouquet)$

Projection

- $\Pi_{\text{bouquet_name}, \text{price}}(\text{bouquet})$

bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
3	Passion	13.0
4	I love you	33.0
5	Pretty in pink	24.5
6	You are beautiful	23.0
7	Amour	20.0
8	Simply red	26.0
9	Be my Valentine	13.5
10	Hello	10.0
11	Ladies Day	12.5
12	To mum with love	15.0
13	Made for mum	20.0
14	Heaven scent	5.0
15	Pastel perfection	8.0
16	Summer sun	22.0
17	Summer breeze	6.0

bouquet_name	price
A New Day	23.5
The flower of love	20.5
Passion	13.0
I love you	33.0
Pretty in pink	24.5
You are beautiful	23.0
Amour	20.0
Simply red	26.0
Be my Valentine	13.5
Hello	10.0
Ladies Day	12.5
To mum with love	15.0
Made for mum	20.0
Heaven scent	5.0
Pastel perfection	8.0
Summer sun	22.0
Summer breeze	6.0

Selection combined with projection

- The Selection and Projection are often combined
- $\Pi_{\text{bouquet_name}, \text{price}}(\sigma_{\text{price} > 20}(\text{bouquet}))$

bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
3	Passion	13.0
4	I love you	33.0
5	Pretty in pink	24.5
6	You are beautiful	23.0
7	Amour	20.0
8	Simply red	26.0
9	Be my Valentine	13.5
10	Hello	10.0
11	Ladies Day	12.5
12	To mum with love	15.0
13	Made for mum	20.0
14	Heaven scent	5.0
15	Pastel perfection	8.0
16	Summer sun	22.0
17	Summer breeze	6.0

Selection

bouquet_id	bouquet_name	price
1	A New Day	23.5
2	The flower of love	20.5
4	I love you	33.0
5	Pretty in pink	24.5
6	You are beautiful	23.0
8	Simply red	26.0
16	Summer sun	22.0

Projection

bouquet_name	price
A New Day	23.5
The flower of love	20.5
I love you	33.0
Pretty in pink	24.5
You are beautiful	23.0
Simply red	26.0
Summer sun	22.0

Writing SQL Commands..

- Most components of an SQL statement are case insensitive, except for literal character data
 - Letters can be typed in either upper or lower case
 - The literal character data must be typed exactly as it appears in the database e.g. If we store a person's surname as 'SMITH' and then search for it using the string 'Smith', the row will not be found
- Readability with indentation and lineation

SQL SELECT Statement

- The purpose of the SELECT statement is to retrieve and display data from one or more database tables
- Extremely powerful command capable of performing the equivalent of the relational algebra's Selection, Projection, and Join operations in a single statement
- SELECT is the most frequently used SQL command

Writing SQL Commands..

- Extended form of BNF (Backus Naur Form) notation will be used to define SQL statements:
 - Upper-case letters represent reserved words
 - Lower-case letters represent user-defined words
 - | indicates a choice among alternatives
 - { }Curly braces indicate a required element
 - []Square brackets indicate an optional element
 - An optional ellipsis (...) indicates optional repetition (0 or more)

SELECT General Form

SELECT [DISTINCT | ALL]

{* | [columnExpression [AS newName]] [,...]}

FROM TableName [alias] [, ...]

[WHERE condition]

[GROUP BY columnList] **[HAVING**
condition]

[ORDER BY columnList]

Processing Sequence

- The sequence of processing in a SELECT statement is:
 - FROM** specifies the table or tables to be used
 - WHERE** filters the rows subject to some condition
 - GROUP BY** forms groups of rows with same column value
 - HAVING** filters the groups subject to some condition
 - SELECT** specifies which columns to appear in the output
 - ORDER BY** specifies the order of the output

Important

- The order of the clauses in the SELECT statement cannot be changed
- The only two mandatory clauses are:
 - **SELECT**
 - **FROM**
- The remainder are optional
- The SELECT operation is closed:
 - The result of a query on a table is another table

Example 1

- List full details of all customers (all rows, columns)

```
SELECT cust_id,name,phone,age,address  
FROM customer;
```

```
SELECT *  
FROM customer;
```



Cust_id	name	phone	age	address
1	Ali Rehman	+4475567855344	21	12 hillwood drive Nottingham ...
2	Ali Rehman	+4475567855344	45	65 oxford drive Nottingham NG...
3	Kristian Liam	+4475567845644	31	176 Groove drive London GH4 ...
4	Lizzi Young	+4474463545024	34	5 Victoria Street Leicester LE6...
5	Ashar Junaid	+4455784186264	65	14 Manor Road Bath BA78 5PA
6	Rebbaca Umar	+4444784553242	23	545 North Street Perth PH80 ...
7	Nyla Noor	<i>NULL</i>	25	38 Church Road Guildford GU4...
8	Ashar Junaid	+4455784186264	56	82 Melbourne Road Nottingha...

Example 2

- Produce a list of for all customer order, showing only order number, date and greeting message(specific columns, all rows)

```
SELECT  
order_id,orderdate,greeting_message  
FROM  
bouquet_order;
```

order_id	orderdate	greeting_message
1	12-12-2023	Happy Birth Day
2	12-12-2023	Happy Birth Day
3	12-01-2024	Happy Birth Day
4	12-08-2022	Happy Mothers Day Mum
5	12-11-2021	Congratulation on new job
6	19-10-2023	Get Well Soon
7	24-05-2021	Happy wedding anniversary
8	23-12-2019	I love you Father
9	26-05-2022	Best wishes for your exams
10	12-09-2022	Congratulations on new home

DISTINCT/ALL

- **DISTINCT** allows you to remove duplicate results
- **ALL** allows you to retain duplicate results
- **ALL** is implied by default

```
SELECT [DISTINCT | ALL]  
{* | [columnExpression [AS newName]] [,...]}  
FROM      TableName [alias] [, ...]  
[WHERE     condition]  
[GROUP BY  columnList] [HAVING condition]  
[ORDER BY  columnList]
```

name
Ali Rehman
Ali Rehman
Kristian Liam
Lizzi Young
Ashar Junaid
Rebbaca Umar
Nyla Noor
Ashar Junaid

SELECT ALL name FROM customer;

SELECT name FROM customer;

SELECT DISTINCT name FROM customer;

name
Ali Rehman
Kristian Liam
Lizzi Young
Ashar Junaid
Rebbaca Umar
Nyla Noor

Alias

- SQL aliases are used to give a table, or a column in a table, a temporary name.
- Aliases are often used to make column names more readable.
- An alias only exists for the duration of that query.
- An alias is created with the AS keyword
- Example:

SELECT name **as** 'Customer Name' **FROM** customer
OR

SELECT name 'Customer Name' **FROM** customer



Customer Name
Ali Rehman
Ali Rehman
Kristian Liam
Lizzi Young
Ashar Junaid
Rebbaca Umar
Nyla Noor
Ashar Junaid

SQL Functions :

String Functions

Function Name	Purpose	Example
	Concat two strings	Select name " " phone from customer
Substr substr(string, start, length)	Find substring	Select substr(name,1,4) from customer
Instr(string, substring)	searches a substring in a string and return the position as integer	Select instr(name, 'a') from customer
length(data)	Give length of string	Select length (name) from customer
lower(string)	converted the string to lowercase	Select lower (name) from customer

String Functions

Function Name	Purpose	Example
upper(string)	converted the string to uppercase	Select upper(name) from customer
ltrim(string,[character])	remove all the characters at the beginning of the string.	SELECT ltrim('£599.99', '£') as result;
rtrim(string,[character])	remove all the characters at the end of the string.	SELECT rtrim('599.99 Pound', 'Pound') as result;
REPLACE(string,pattern,replacement)	replace all occurrences of a specified string with another string.	Select replace(phone, '+44', '0')from customer
trim(string, character)	removes specified characters at the beginning and the end of a string.	SELECT trim('£599.9£', '£') as result;

COMP1004

Databases and Interfaces

Module conveners:

Nazia Hameed, Kai Xu